SVKM'S
NMIMS
Deemed-to-be UNIVERSITY

Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

IT DEPARTMENT

Project Synopsis

Title of Project:

Traceveil: AI-Assisted Real-Time Smart Home IoT Digital Forensics and Incident Response Platform

Team Members:

Atharva Rangnekar - K053 | Aarya Rao - K054 | Ayra Jha - K028

Under the supervision of

Prof. Rohit Suryawanshi

Table of Contents

1. Title of the Project

Traceveil: AI-Assisted Real-Time Smart Home IoT Digital Forensics and Incident Response Platform

2. Team Members

1. Atharva Rangnekar (K053)
2. Aarya Rao (K054)
3. Ayra Jha (K028)

Project Guide: Prof. Rohit Suryawanshi

3. Introduction

Smart homes combine cameras, door sensors, voice assistants, smart plugs, thermostats, gateways and embedded controllers that produce network records, telemetry, authentication traces and state changes. During an incident, investigators may need both retrospective analysis of stored evidence and immediate awareness of incoming device activity. Relevant evidence remains distributed across devices and expressed through different schemas, clocks and identifiers, so a single alert rarely provides a defensible account.

Traceveil is an academic hybrid digital-forensics and incident-response prototype. It accepts public IoT datasets, simulated smart-home evidence and controlled live telemetry from an authorized laboratory setup. Batch and live records are validated, normalized into the same canonical event model and processed by the same deterministic investigation pipeline. Accepted live events are retained as evidence so an incident detected in near real time can later be correlated, reconstructed and reviewed through the forensic workflow. Traceveil does not replace an enterprise SIEM, an IoT security platform or a certified forensic suite.

3.1. Problem Statement

Public IoT datasets are useful for repeatable experiments, but their fields and labels differ and they do not directly provide a case-oriented forensic view. Academic systems often emphasize either offline dataset classification or real-time intrusion detection, leaving a separation between immediate alerting and post-incident reconstruction. Traceveil addresses this practical gap by allowing controlled live telemetry and historical evidence to converge into one canonical model, where a qualifying event can generate an alert and remain preserved for later correlation, timeline reconstruction and review. Generative AI creates an additional reproducibility risk; therefore Python remains the only computational authority and the local model is confined to evidence-grounded narration and validated layout planning.

4. Literature Survey

4.1. IoT DFIR and Public Datasets

NIST describes digital forensics as a capability that supports incident response through collection, examination, analysis and reporting [1]. In IoT environments, heterogeneity, limited storage, volatile evidence and clock inconsistency make these activities difficult [4]. A sound prototype must preserve the raw-record link, distinguish observed time from import time, document normalization decisions and avoid treating correlation as proof of causation.

TON_IoT contains heterogeneous IoT/IIoT telemetry, operating-system traces and network traffic captured in a cyber-range environment [2]. CICIoT2023 provides a large IoT attack benchmark with benign and malicious traffic from numerous devices and attack scenarios [3]. Traceveil uses these datasets to test batch adapters, rules and performance, uses small simulated cases for exact correlation and timeline tests, and adds controlled laboratory telemetry for live-ingestion and alert-delivery evaluation. These sources and test devices support academic evaluation but do not establish production readiness or legal admissibility.

Rule-based logic is retained because each alert can show its condition and contributing records. Behavioural baselines add deviations in rates, peers, protocols or active periods. Risk is a bounded, versioned combination of stored factors such as severity, confidence, repetition, device criticality and cross-device corroboration. NetworkX represents device/entity relationships, while stable sorting and explicit correlation windows produce reproducible graphs and timelines. NIST log-management guidance supports structured, maintained security-log processes [5].

4.2. Comparison with Existing Platforms

Platform	Enterprise Strength	Traceveil Position
Microsoft Defender for IoT	IoT/OT discovery, network sensors, behavioural analytics, vulnerability information and Microsoft ecosystem integration [8].	Academic hybrid DFIR prototype for uploaded evidence and controlled laboratory telemetry with transparent deterministic derivation.
AWS IoT Device Defender	AWS IoT configuration audits, device behaviour metrics and mitigation actions [9].	Cloud-independent case analysis and controlled live telemetry; no fleet enforcement or autonomous mitigation.
Splunk Enterprise Security	Broad ingestion, correlation searches, risk-oriented triage and incident workflows [10].	Purpose-built canonical IoT event model shared by batch evidence and laboratory live events.
IBM QRadar	Event/flow normalization, correlation rules and prioritized offenses [11].	Smaller case reconstruction, live-alert demonstration, timelines, device graphs and bounded local AI.

These products are mature enterprise systems with deployment, scale, content and integration capabilities outside Traceveil's scope. The comparison is therefore contextual rather than competitive: Traceveil demonstrates selected DFIR concepts transparently using academic data.

4.3. Research Gap

A gap exists between intrusion-detection experiments and complete, inspectable DFIR workflows. Many prototypes classify rows or raise real-time alerts but do not preserve a traceable path through normalization, risk factors, cross-device links, timeline membership and later evidence review. Adaptive dashboards may also allow generative systems to produce unchecked content. Traceveil demonstrates a unified academic pipeline in which historical evidence and controlled live telemetry use the same canonical data model and deterministic analytics. It combines reproducible alerting, evidence preservation and post-incident reconstruction without claiming novelty beyond the scope supported by the existing literature.

5. Why is this particular topic chosen?

Smart-home IoT was selected because it is familiar yet forensically demanding. Several vendors and device types interact in one private environment, so evidence is fragmented and privacy-sensitive. Public datasets permit repeatable experiments, simulated incidents create controlled multi-device narratives, and a small authorized IoT laboratory provides a practical way to demonstrate live telemetry acquisition, near-real-time alerting, evidence preservation and incident reconstruction without collecting unrelated household traffic. The domain supports the study of explainability, correlation, privacy-conscious local AI and safe adaptive visualization within an academic testbed.

6. Objective and Scope of the project

6.1. Objectives and Scope

- Ingest selected TON_IoT, CICIoT2023, simulated cases and uploaded evidence with source hashes, adapter versions and validation errors.
- Receive validated live telemetry from an Arduino-compatible source and IoT test endpoint in a controlled, authorized laboratory setup.
- Normalize batch and live records into one canonical event model while retaining raw-record, device, source and ingestion provenance.
- Use deterministic Python for filtering, stable sorting, rules, baselines, bounded risk scoring, correlation, timelines, chart aggregates and graph values.
- Generate live alerts only when deterministic detection conditions are satisfied.
- Preserve accepted live events and resulting alerts as evidence for later investigation and timeline reconstruction.
- Provide a React dashboard with cases, evidence, metrics, live status, event and alert feeds, timelines, charts, tables, device graphs and findings panels.
- Use local Qwen 9B Q4_K_M through Ollama only for summaries, grounded explanations, investigation chat, email drafting and visualization layout planning.
- Evaluate batch correctness, reproducibility, detection and correlation quality, visualization safety, live ingestion, real-time delivery, usability and performance under declared conditions.

In scope are batch imports, case management, deterministic analysis, controlled laboratory live IoT ingestion, Arduino-compatible telemetry, near-real-time alert demonstration, dashboard review, preservation of live evidence, local AI assistance, SQLite storage, investigator-approved SMTP alerts and report export. Controlled suspicious scenarios will be performed only against owned or authorized test devices in an isolated or approved environment. Out of scope are production-scale live interception, passive capture of arbitrary third-party networks, unauthorized device access, firmware extraction, production fleet management, autonomous containment or attack execution, legal-admissibility claims and replacement of enterprise products.

6.2. Architecture and Workflow

The layered architecture supports two evidence input paths: batch adapters for public datasets, simulated cases and uploaded files; and a minimal live IoT collection layer for controlled device telemetry. Both paths pass through validation and normalization into one canonical event model and deterministic DFIR engine. FastAPI and Pydantic validate contracts, SQLite stores cases and derived artifacts through a repository boundary, and planned SSE or WebSocket delivery will forward verified events and alerts without calculating forensic values. The optional visualization and AI layer consumes only validated backend outputs.

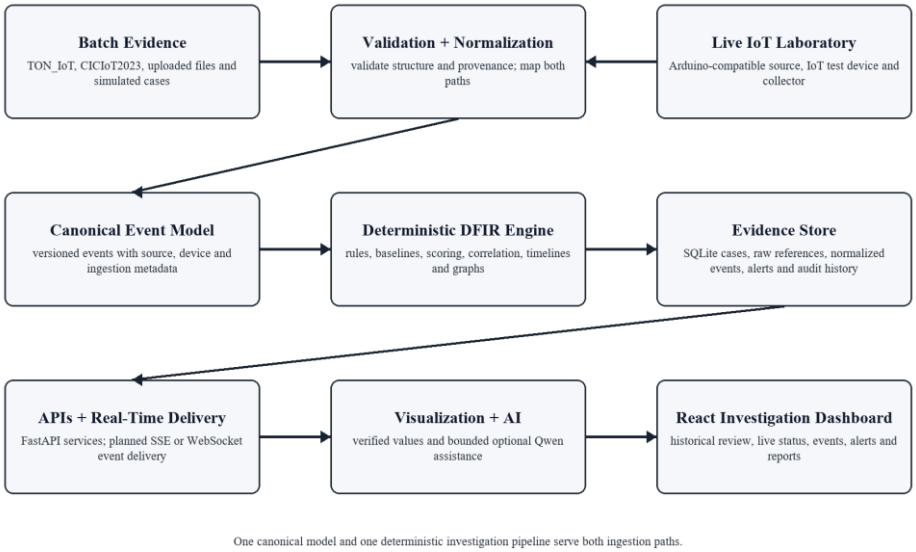

Figure 1. Proposed Traceveil hybrid batch/live system architecture

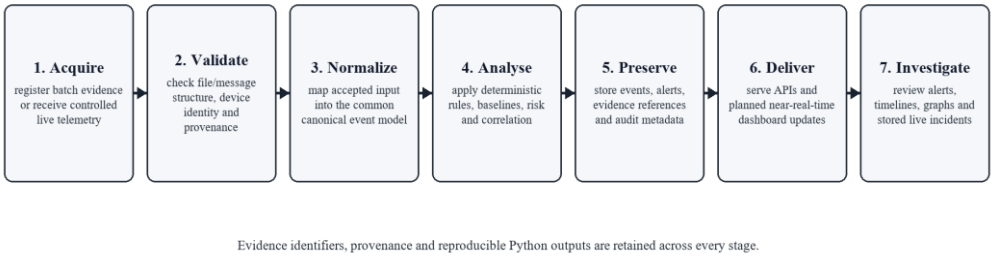

Figure 2. Unified evidence-to-investigation workflow

The workflow registers and hashes batch evidence or receives controlled live telemetry through a validated collector. It checks structure and provenance, maps accepted inputs into canonical events, applies deterministic rules, baselines, scoring and correlation, preserves evidence and derived artifacts, and delivers verified results through APIs and planned real-time updates. Every artifact records evidence identifiers and configuration versions. The model is invoked only after deterministic analysis; Ollama failure cannot disable ingestion, alerting, persistence or investigation functions.

6.3. Module Descriptions

Module	Responsibility
Batch Evidence and Adapter Layer	Registers cases and hashes; validates uploaded files and dataset columns; records source metadata and documented rejections.
Live IoT Collection Layer	Receives controlled device telemetry; validates message structure and source identity; records ingestion metadata; forwards accepted input to normalization.
Canonical Event and Normalization Layer	Maps batch and live inputs into one versioned schema while preserving raw-record, device, source and ingestion provenance.
Detection and Risk Engine	Executes deterministic rules and baselines; returns evidence lists, condition traces, factor breakdowns and bounded scores.
Correlation and Timeline Engine	Links events using configured keys and windows; records link reasons; stable-sorts historical and incoming case chronology.
Graph and Query Engine	Uses NetworkX and Pandas for graph measures, filters, aggregates and chart-ready values.
API, Storage and Real-Time Delivery	Provides FastAPI/Pydantic services, SQLite repositories, audit history, exports and planned event/alert delivery; calculates no forensic values.
Visualization and AI Layer	Renders allow-listed components and provides bounded, evidence-grounded local assistance over verified historical or live results.
React Investigation Dashboard	Presents cases, evidence, live status, events, alerts, timelines, graphs, charts, findings, chat and reports.

6.4. Visualization Engine

The Visualization Engine maps a Pydantic-validated JSON specification to predefined React components implemented with Recharts and React Flow. Components may update from verified live backend values, but the specification remains versioned and allow-listed and may reference only deterministic data and existing fields. It cannot contain JavaScript, JSX, SQL, Python, free-form expressions or numerical arrays invented by the model. Unknown components, fields or case references are rejected and a deterministic fallback layout is used. The model must never invent live measurements, alert counts, risk scores, chart values or graph data.

6.5. AI Investigation Assistant

Qwen 9B Q4_K_M runs locally through Ollama. Structured-output support allows schema-constrained JSON [7], with Pydantic validation and evidence-identifier checks added by Traceveil. The assistant may summarize deterministic findings, explain a stored rule trace, correlation path or already-detected live incident, answer questions from selected evidence, draft an email for human approval and plan component placement. It cannot decide independently that an attack occurred, create alerts, calculate scores, send mail autonomously, execute code or attack logic, modify evidence, or present an unsupported inference as fact. Prompts and responses are logged with model, template version and evidence identifiers. The literature recognizes both opportunities and reliability/privacy challenges for LLMs in cybersecurity [6].

6.6. Testing and Evaluation

Area	Method and Measure
Normalization	Boundary and malformed fixtures; exact normalized values and explicit rejection reasons for batch and live inputs.
Reproducibility	Repeated golden runs; exact equality of order, scores, correlations, timelines and graph data.
Detection	Labeled dataset subsets and controlled live suspicious scenarios; rule outcomes and applicable precision, recall, F1-score and confusion matrix, with limitations reported.
Correlation	Curated multi-device cases; incident grouping, link precision/recall and ordered-event agreement.
Live ingestion	Valid and malformed telemetry, device/source identification, ingestion timestamps, provenance and event persistence.
Real-time behaviour	Measured event/alert delivery latency, temporary disconnections and safe reconnect behaviour under declared hardware and network conditions.
Visualization/AI	Unknown components, prompt injection, absent evidence and malformed JSON; unsafe output rejected and references resolved.
Integration/Performance	API, persistence, SMTP mock and user tasks; latency, throughput and memory reported with test conditions.
End-to-end demonstration	Physical event, telemetry receipt, canonical normalization, deterministic alert, dashboard update, evidence persistence and timeline reconstruction.

Acceptance emphasizes correctness, traceability and measured behaviour rather than exaggerated accuracy or latency claims. Dataset and live results will be reported as prototype measurements under declared samples, hardware, network and configuration. A disabled or unavailable Ollama service must leave batch/live ingestion, deterministic analysis, alerting, correlation, timelines, graphs and manual layouts usable.

6.7. Security, Limitations, Future Scope and Conclusion

Evidence is processed locally where possible; uploads and live messages are type, size or structure checked; paths and identifiers are normalized; secrets are excluded from model context; and case actions are audited. The laboratory collector receives only authorized test-device telemetry. The interface distinguishes deterministic findings from AI narrative and avoids attributing identity or intent solely from an anomaly, address or correlation score.

The main limitation is the controlled academic scale: public testbeds, simulations and a small laboratory setup cannot reproduce every proprietary protocol, clock error, vendor cloud, household context or production workload. A minimal controlled collector is now in scope; broader MQTT compatibility, additional IoT protocols, gateway-level and secure remote collectors, larger device fleets, stronger device authentication, production message brokers, clock-drift estimation, richer entity resolution, PostgreSQL, role-based access control, immutable evidence storage, SIEM/SOAR export and controlled user studies remain future work. Traceveil concludes as a privacy-conscious academic demonstration in which Python calculates every forensic and visual value while local AI improves readability only within a bounded, validated role.

7. Hardware and Software Requirements

7.1. Hardware

- 64-bit quad-core processor; 16 GB RAM minimum (24-32 GB recommended for local inference).
- 30 GB free SSD space; optional supported GPU.
- Arduino-compatible microcontroller for the controlled live demonstration.
- At least one IoT test device or controllable IoT endpoint.
- Local Wi-Fi or network environment isolated or authorized for the demonstration.
- Supporting data cables and power as required by the selected hardware.

The minimum computer configuration supports development, deterministic dataset analysis and CPU-based model execution on representative samples. The live demonstration additionally requires an Arduino-compatible microcontroller, an IoT test endpoint and an authorized local network; exact hardware remains to be selected. Additional memory is recommended because Pandas may hold normalized records and derived tables in memory while Ollama loads the quantized model. A supported GPU improves response time but is not required for correctness, and all core forensic modules remain usable when the model is disabled.

7.2. Software

- Frontend: React, Vite, Tailwind CSS, shadcn/ui, Recharts and React Flow.
- Backend: Python, FastAPI, Pandas, NetworkX and Pydantic.
- Database and AI: SQLite, optional future PostgreSQL, Ollama with Qwen 9B Q4_K_M.
- Planned live communication: MQTT and/or HTTP telemetry ingestion, with SSE or WebSockets for server-driven frontend updates.
- Alerts and development: SMTP, Git, GitHub, Codex, Pytest and a modern browser.

The frontend is responsible only for presentation and validated user interaction. FastAPI will expose case, evidence, analysis, live-ingestion, visualization and assistant contracts through Pydantic. Pandas performs schema transformation and aggregation, while NetworkX builds correlation graphs and returns calculated nodes, edges and metrics. MQTT and/or HTTP will be selected for controlled telemetry ingestion, and SSE or WebSockets will be selected for server-driven updates based on implementation needs; no protocol-specific library is claimed as implemented. SQLite is sufficient for a single-user academic prototype, while the repository boundary permits later PostgreSQL migration.

Ollama runs locally and is called through a backend service that supplies bounded evidence context. SMTP credentials are stored outside the source repository and email is sent only after explicit investigator approval. Git and GitHub maintain the project history, automated tests protect deterministic behaviour, and Codex may assist implementation and review without becoming part of the runtime evidence pipeline.

8. Final Deliverables

- Documented React/FastAPI source repository and reproducible setup instructions.
- Batch evidence adapters, canonical event schema and representative smart-home cases.
- Controlled live IoT telemetry collector with validated device/source metadata.
- Arduino-compatible and IoT test-device laboratory demonstration setup.
- Deterministic Python detection, scoring, correlation, timeline, chart and graph modules for batch and live events.
- Dashboard with historical investigation views, live status, real-time event updates and deterministic live alerts.
- Validated Visualization Engine, bounded local AI assistant, SQLite case history and investigator-approved SMTP support.
- Persisted live incident evidence, automated tests, evaluation results, exports and final report.
- One complete batch demonstration case and one complete controlled live suspicious-event case.

Deliverable Area	Acceptance Evidence
Batch evidence pipeline	Successful import logs, documented field mappings, hashes and canonical-schema validation.
Live IoT pipeline	Validated telemetry receipt, device/source provenance, canonical normalization and persisted evidence.
Deterministic analytics	Golden-case matches and controlled live rule outcomes for risk factors, graph edges, chart values and timelines.
Real-time dashboard	Verified event, alert, device-status and timeline updates using backend-provided values.
Visualization and AI	Allow-listed JSON tests, invalid-spec rejection and proof that AI cannot modify computed values.
Reporting and privacy	Auditable exports, investigator-approved email flow, secret handling and local case-data cleanup.
Physical demonstration	Normal telemetry, controlled suspicious scenario, deterministic alert, persistence and later forensic review.

The submitted implementation package will include sample configuration, database initialization, supported dataset-field mappings, live telemetry contracts, hardware setup notes and representative simulated cases. Each analytical result will expose the evidence identifiers and configuration version needed to reproduce it. The dashboard will demonstrate complete investigation paths for both imported evidence and accepted live telemetry, including alerts, risk factors, cross-device links, timeline review and export.

Delivery acceptance will require successful automated tests, schema validation for every batch, live-ingestion, API and visualization response, a clean installation and hardware walkthrough, repeatable golden-case results, and an authorized controlled live demonstration. Performance will be measured under declared hardware and network conditions rather than guaranteed. Source files, documentation and demonstration data will exclude passwords, SMTP secrets, unrelated household telemetry and model-generated claims unsupported by evidence.

Two demonstration cases will accompany the submission. The batch case will contain a controlled sequence of simulated events with expected normalized records, rule matches, risk factors, correlation edges and timeline. The live case will show normal telemetry followed by an authorized suspicious scenario, deterministic alerting, evidence persistence and subsequent forensic review. Reviewers will be able to inspect provenance and confirm that disabling the language model does not alter any forensic value.

1. Normal device telemetry is visible.
2. A controlled suspicious scenario occurs against an owned or authorized test device.
3. Traceveil receives and validates the telemetry.
4. A deterministic detection rule triggers.
5. A live alert appears on the dashboard.
6. The accepted events and alert are preserved as evidence.
7. The incident timeline and device/entity relationships update from verified backend values.
8. The captured incident remains available for later forensic review.

The evaluation bundle will preserve the configuration used for each run: adapter version, live-message contract version, rule-set version, baseline window, scoring weights, correlation window, dataset subset, selected test hardware, network conditions and software environment. Reported performance figures will distinguish measured import, ingestion, analysis, delivery, query and rendering times from qualitative observations about local-model responsiveness. Screenshots and exports will use pseudonymous device identifiers. Any discrepancy between expected and observed results will be treated as a defect rather than adjusted through an AI-generated explanation.

9. References

[1] K. Kent, S. Chevalier, T. Grance, and H. Dang, Guide to Integrating Forensic Techniques into Incident Response, NIST SP 800-86, 2006, doi: 10.6028/NIST.SP.800-86.

[2] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems," IEEE Access, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[3] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," Sensors, vol. 23, no. 13, Art. 5941, 2023, doi: 10.3390/s23135941.

[4] A. O. Akinbi, "Digital Forensics Challenges and Readiness for 6G Internet of Things (IoT) Networks," WIREs Forensic Science, vol. 5, no. 6, e1496, 2023, doi: 10.1002/wfs2.1496.

[5] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92, 2006, doi: 10.6028/NIST.SP.800-92.

[6] H. Xu et al., "Large Language Models for Cyber Security: A Systematic Literature Review," CoRR, abs/2405.04760, 2024, doi: 10.48550/arXiv.2405.04760.

[7] Ollama, "Structured Outputs," Ollama Documentation. [Online]. Available: https://docs.ollama.com/capabilities/structured-outputs. Accessed: Jul. 14, 2026.

[8] Microsoft, "Defender for IoT - OT Architecture and Components," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/defender-for-iot/organizations/architecture. Accessed: Jul. 14, 2026.

[9] Amazon Web Services, "What is AWS IoT Device Defender?" AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/iot-device-defender/latest/devguide/what-is-device-defender.html. Accessed: Jul. 14, 2026.

[10] Splunk, "Correlation Search Overview for Splunk Enterprise Security," Splunk Documentation. [Online]. Available: https://help.splunk.com/en/splunk-enterprise-security-7/administer/7.3/correlation-searches/correlation-search-overview-for-splunk-enterprise-security. Accessed: Jul. 14, 2026.

[11] IBM, "Custom Rules in IBM QRadar," IBM Documentation. [Online]. Available: https://www.ibm.com/docs/en/qradar-on-cloud?topic=siem-custom-rules. Accessed: Jul. 14, 2026.